



9/1/2026

N + - Network Fundamentals & Building Networks.

Authentication and Access Control.

TASK 1.

- ❖ Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

➤ Mock Zomato Admin Login Form:

HTML + JavaScript Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Zomato Admin Login</title>
</head>
<body>

  <h2>Zomato Admin Panel</h2>

  <input type="text" id="username" placeholder="Username"><br><br>
  <input type="password" id="password"
placeholder="Password"><br><br>

  <button onclick="login()">Login</button>

  <p id="result"></p>

  <script>
    function login() {
      const adminUsername = "admin";
      const adminPassword = "zomato123";

      const username = document.getElementById("username").value;
      const password = document.getElementById("password").value;
```

```

        if (username === adminUsername && password === adminPassword)
        {
            document.getElementById("result").innerText = "Access Granted";
        } else {
            document.getElementById("result").innerText = "Access Denied";
        }
    }
}
</script>

</body>
</html>

```

Result: The form displays “Access Granted” only when the entered username and password match the hardcoded admin credentials.

TASK 2.

❖ Write a Python function `check_access(user_role, resource)` that returns True if the `user_role` is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'. Use a dictionary to map roles to allowed resources.

➤ Role-Based Access Control:

Python Code:

```

def check_access(user_role, resource):
    allowed_resources = {
        "customer": ["order_history"],
        "restaurant_owner": ["menu_edit", "order_history"],
        "admin": ["menu_edit", "order_history", "admin_dashboard"]
    }

```

```
return resource in allowed_resources.get(user_role, [])
```

```
# Test cases
```

```
print(check_access("customer", "order_history"))  
print(check_access("customer", "admin_dashboard"))  
print(check_access("restaurant_owner", "menu_edit"))  
print(check_access("restaurant_owner", "admin_dashboard"))
```

Output:

```
True  
False  
True  
False
```

Result: The function returns **True** when the role has permission for the resource and **False** otherwise.

TASK 3.

❖ **Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.**

➤ **Two-Factor Authentication (2FA):**

Python Code:

```
import random  
  
password = input("Enter password: ")
```

```

if password == "paytm123":
    otp = random.randint(100000, 999999)
    print("Your OTP is:", otp)

    user_otp = int(input("Enter OTP: "))

    if user_otp == otp:
        print("Login Successful")
    else:
        print("Invalid OTP")
else:
    print("Incorrect Password")

```

Result: A 6-digit OTP is generated after the correct password is entered, and “Login Successful” is displayed only when the entered OTP matches.

TASK 4.

❖ Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days, Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.

➤ Filter Premium Users:

JavaScript Code:

```

function getActivePremiumUsers(users) {
    const today = new Date("2024-06-10");
    const sevenDaysAgo = new Date("2024-06-03");

    return users.filter(user => {

```

```
const lastLogin = new Date(user.lastLogin);

return user.isPremium &&
    lastLogin >= sevenDaysAgo &&
    lastLogin <= today;
});
}

// Example users
const users = [
    { username: "Aman", isPremium: true, lastLogin: "2024-06-08" },
    { username: "Riya", isPremium: false, lastLogin: "2024-06-09" },
    { username: "Karan", isPremium: true, lastLogin: "2024-05-25" },
    { username: "Neha", isPremium: true, lastLogin: "2024-06-05" }
];

console.log(getActivePremiumUsers(users));
```

Output:

Aman
Neha

Result: The function returns only users who are **premium** and have logged in within the **last 7 days**.

TASK 5.

❖ Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

➤ **Common Access Control Mistakes:**

Mistake	Fix / Prevention Tip
1. Giving users more permissions than needed	Follow the principle of least privilege and give only required access.
2. Not checking permissions on every request	Always verify the user's role and permission before allowing access.
3. Using predictable user IDs or URLs	Check authorization on the server instead of trusting the URL or user ID.
4. Forgetting to remove access after a role change	Update or revoke permissions immediately when a user's role changes.
5. Relying only on frontend security	Perform all important access-control checks on the server side.